# PRD: Points & Rewards Service (Interview Showcase)

Oct 26, 2026 · @Steven Goh

## Overview and goals

Build a Java and Spring Boot points and rewards backend that proves, with working code and tests, every skill the Rakuten INPD Senior Java Backend Engineer role asks for. Users earn points from purchases, campaigns multiply or add points, and users redeem points. The domain mirrors an incentive platform, so it doubles as an interview talking point.

The project succeeds when:

- Every required skill in the job ad maps to a feature that runs and is tested.
- Two concurrent redeem requests can never spend the same points, proven by an automated test.
- The service runs as 3 replicas on local Kubernetes, and the problems that creates are solved and explained.
- The README explains each design decision and its trade-off in one or two lines.
- Load test results show the effect of caching with real, measured numbers.

## Scope

The backend is the priority; a frontend is optional and comes only after milestone 4.

| Area | In scope | Out of scope |
|---|---|---|
| Core | Earn, redeem, balance, history, campaigns, points expiry | Real payment processing, real merchants |
| Users | Registration, login, user and admin roles | Social login, password reset emails |
| Security | JWT, role-based access, idempotency, rate limiting, audit log | External identity provider, MFA |
| Infra | Docker, docker-compose, Kubernetes on Docker Desktop, GitHub Actions | Real cloud cluster (AWS is a stretch goal) |

| Area | In scope | Out of scope |
|---|---|---|
| Showcase | Redis cache, SSE, outbox with a message queue, ShedLock, Resilience4j | Multi-region, service mesh |
| Frontend | Simple dashboard, later | Mobile app |

## Domain model

Points are never stored as a single number that gets overwritten; every change is a row in an append-only ledger, and the balance is derived from it.

| Entity | Purpose | Key fields |
|---|---|---|
| User | Account that owns points | id, email, password hash, role (USER, ADMIN) |
| PointAccount | One per user; holds the cached balance and a version for locking | user_id, balance, version |
| LedgerEntry | Append-only record of every earn, redeem, expiry or reversal | id, account_id, type, amount (signed), reference_id, campaign_id, expires_at, created_at |
| Campaign | Rule that changes how many points a purchase earns | id, name, rule_type, params (JSON), start_at, end_at, active |
| IdempotencyRecord | Stores the result of a request so a retry returns the same answer | key, user_id, request_hash, response, created_at |
| OutboxEvent | Event saved in the same transaction as the ledger write, published later | id, type, payload, status, created_at |
| AuditLog | Who did what and when, for admin and money-related actions | actor_id, action, target, details, created_at |

Business rules:

1. Earning: base rate is 1 point per 100 currency units spent; active campaigns are applied on top.
2. Campaign rule types: multiplier (for example 2x), fixed bonus, and category bonus. When several apply, the service picks the best one for the user, not all of them.
3. Redeeming: a user can only redeem up to their available balance. The balance can never go below zero.
4. Expiry: earned points expire 12 months after they are earned. Redemption uses the oldest points first.
5. Reversal: a refund creates a negative ledger entry linked to the original reference; entries are never edited or deleted.

## Functional requirements

All endpoints live under `/api/v1`, return JSON, and use one consistent error format (RFC 7807 Problem Details). List endpoints are paginated.

| Method | Endpoint | Who | What it does |
|---|---|---|---|
| POST | /auth/register | Public | Create a user account |
| POST | /auth/login | Public | Return an access token and a refresh token |
| POST | /auth/refresh | Public | Exchange a refresh token for a new access token |
| POST | /points/earn | User (or internal service) | Record a purchase and award points; requires an Idempotency-Key header |
| POST | /points/redeem | User | Spend points; requires an Idempotency-Key header |
| GET | /points/balance | User | Current available balance |
| GET | /points/history | User | Paginated ledger entries, newest first |
| GET | /points/stream | User | SSE stream of live balance changes |
| POST | /admin/campaigns | Admin | Create a campaign |

| Method | Endpoint | Who | What it does |
|---|---|---|---|
| PUT | /admin/campaigns/{id} | Admin | Update a campaign |
| GET | /admin/campaigns | Admin | List campaigns with filters |
| POST | /admin/points/adjust | Admin | Manual credit or debit with a required reason; written to the audit log |
| GET | /admin/audit-logs | Admin | Paginated audit log |

Acceptance rules:

- Every write that touches points is one database transaction: ledger entry, balance update and outbox event commit together or not at all.
- Validation errors return 400 with field-level messages; missing auth returns 401; wrong role returns 403; insufficient points returns 422.
- OpenAPI docs are generated with springdoc and available at `/swagger-ui`.

## Security requirements

Security gets the most time because the job ad lists it as a core requirement.

- [ ] **Authentication:** Spring Security with stateless JWT (short-lived access token, longer refresh token). Passwords hashed with BCrypt or Argon2.
- [ ] **Authorization:** role-based access (USER, ADMIN) with `@PreAuthorize`. A user can only read or change their own account; test that user A cannot see user B's history (broken access control).
- [ ] **Idempotency:** every earn and redeem request needs an `Idempotency-Key` header. A repeated key returns the stored response; the same key with a different body returns 409.
- [ ] **Input validation:** Bean Validation on every request; amounts must be positive and within limits.
- [ ] **SQL injection:** only parameterised queries (JPA or named parameters), with a test that tries an injection string.
- [ ] **Rate limiting:** per-user limit on earn and redeem (Bucket4j backed by Redis, so it works across 3 replicas). Over the limit returns 429.

- [ ] **Secrets:** no secrets in code or Git; read from environment variables locally and Kubernetes Secrets in the cluster.
- [ ] **Audit log:** every admin action and manual adjustment is recorded with who, what and when.
- [ ] **Safe errors:** responses never leak stack traces, SQL or internal class names.
- [ ] **Security headers and CORS:** strict CORS allow-list, standard security headers on.
- [ ] **Dependency scanning:** OWASP Dependency-Check or GitHub Dependabot in CI.

## Data and database requirements

PostgreSQL is the source of truth; Redis only holds data that can be rebuilt from it.

- **Migrations:** every schema change is a Flyway migration; the app never relies on Hibernate auto-DDL.
- **Concurrency:** redeem uses optimistic locking ( `@Version` on PointAccount) with a retry, and a second version uses `SELECT ... FOR UPDATE`, so you can compare both in the interview.
- **Constraints:** a database CHECK keeps balance at zero or above; a UNIQUE constraint on (user_id, idempotency_key) is the last line of defence against duplicates.
- **Indexes:** on ledger_entry(account_id, created_at) for history, on ledger_entry(expires_at) for the expiry job, and on campaign(active, start_at, end_at).
- **Query checks:** run `EXPLAIN ANALYZE` on history and expiry queries with at least 1 million seeded ledger rows, and note the result in the README.
- **Avoid N+1:** history and campaign listings use fetch joins or DTO projections; a test counts queries to prove it.
- **Money and points:** stored as integer types ( `BIGINT` ), never floating point.
- **Seed data:** a script that creates test users, campaigns and a large ledger for load testing.

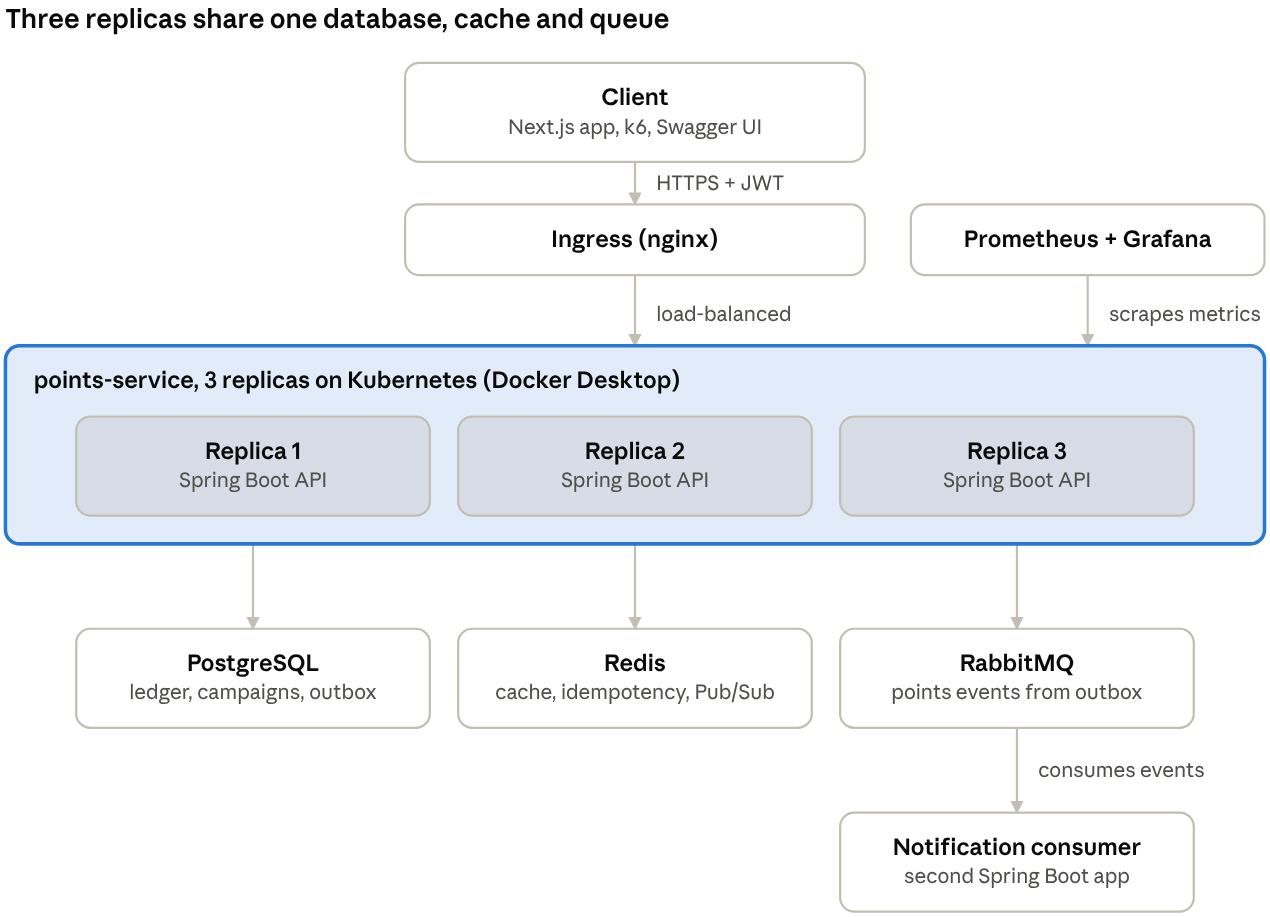
## Showcase features

Each extra is tied to a real problem that appears once the service runs as 3 replicas; that problem is what you explain in the interview.

| Feature | Problem it solves | Requirement |
|---|---|---|
| Redis cache | Balance and active campaigns are read far more often than they change | Cache balance per user and the active campaign list; evict on every ledger write and campaign change; a short TTL as a safety net |
| Redis idempotency store | A retried request can land on a different replica | Idempotency records shared in Redis, with the database unique constraint as backup |
| SSE live balance | A user's stream is connected to only one replica | `/points/stream` per user; balance changes published through Redis Pub/Sub so every replica can push to its own connected users; heartbeat every 15 seconds |
| Transactional outbox | The app can crash after saving to the database but before sending an event | Outbox row written in the same transaction; a relay publishes to RabbitMQ (or Kafka); consumers are idempotent |
| Notification consumer | Shows event-driven design across services | A small second Spring Boot app consumes PointsEarned and PointsRedeemed and logs or stores a notification |
| ShedLock | The points-expiry job would run on all 3 replicas | Scheduled expiry job runs on exactly one replica at a time |
| Resilience4j | A slow external partner can block the service | A mock partner-verification call with timeout, retry and circuit breaker; the earn flow still works when the partner is down (queued for later) |
| Design patterns | Campaign logic would become one large if-else | Strategy per rule type, a Factory to choose it, and Spring events (Observer) for side effects |

# Infrastructure

The whole system runs locally twice: with docker-compose for daily work, and on Docker Desktop's built-in Kubernetes with 3 replicas for the scaling showcase.




system architecture · 3 replicas, shared stores

All three replicas are stateless; anything they must share (data, cache, locks, live events) lives in PostgreSQL, Redis or RabbitMQ.

### Docker

- Multi-stage Dockerfile (build with Maven or Gradle, run on a slim JRE image) as a non-root user.
- `docker-compose.yml` starts the app, PostgreSQL, Redis, RabbitMQ, the notification consumer, Prometheus and Grafana with one command.

### Kubernetes (Docker Desktop)

- Deployment for points-service with 3 replicas, plus a Service; an Ingress (ingress-nginx) in front.

- Deployments or StatefulSets for PostgreSQL, Redis and RabbitMQ (fine for local; in production these would be managed services).
- Liveness and readiness probes on Spring Boot Actuator health endpoints, and graceful shutdown so in-flight requests finish.
- ConfigMap for settings and Secret for passwords and the JWT key.
- CPU and memory requests and limits on every container.
- Rolling update demo: deploy a new version under load with zero failed requests.
- Optional: Horizontal Pod Autoscaler with metrics-server.

**CI (GitHub Actions)**

- On every push: build, unit and integration tests (Testcontainers), dependency scan, and Docker image build.
- Branch protection on main; clear commit messages and pull requests, even when working alone.

## Observability and testing

Every claim in the README must be backed by a test or a measured number.

**Observability**

- Metrics: Micrometer to Prometheus, with a Grafana dashboard for request rate, latency (p50 and p95), error rate, cache hit rate and outbox lag.
- Logs: structured JSON logs with a correlation ID passed from the request through to the queue consumer.
- Tracing (optional): OpenTelemetry traces across points-service and the notification consumer.

**Testing**

- Unit tests with JUnit 5 and Mockito for campaign strategies, balance rules and expiry.
- Integration tests with Testcontainers (PostgreSQL, Redis, RabbitMQ), no in-memory database.
- Concurrency test: 50 parallel redeem requests against a balance that only covers one; exactly one succeeds and the balance never goes negative.
- Idempotency test: the same request sent 10 times creates exactly one ledger entry.
- Security tests: no token, wrong role, another user's data, and an injection attempt all fail correctly.

- Target 80% or more line coverage on the service layer, reported by JaCoCo in CI.

**Load test (k6)**

- Scenario: mostly balance reads with some earns and redeems, against the 3-replica cluster.
- Run once with the cache off and once with it on; record requests per second and p95 latency for both in the README.

## Tech stack

| Layer | Choice |
|---|---|
| Language and framework | Java 21, Spring Boot 3 (Web, Security, Data JPA, Validation, Actuator) |
| Database | PostgreSQL 16, Flyway |
| Cache and coordination | Redis (cache, idempotency, rate limits, Pub/Sub), ShedLock |
| Messaging | RabbitMQ (Kafka as an alternative if you want to learn it) |
| Resilience | Resilience4j, Bucket4j |
| API docs | springdoc-openapi |
| Testing | JUnit 5, Mockito, Testcontainers, JaCoCo, k6 |
| Observability | Micrometer, Prometheus, Grafana, OpenTelemetry (optional) |
| Build and CI | Maven or Gradle, GitHub Actions |
| Containers | Docker, docker-compose, Kubernetes on Docker Desktop |
| Frontend (later) | Next.js with TypeScript |

# Milestones and acceptance criteria

Build in this order; each milestone must be done and tested before the next starts. Milestones 1 to 3 are the minimum to show in an interview.

### M1. Core points engine

- [ ] Earn, redeem, balance and history endpoints work against PostgreSQL with Flyway.
- [ ] Ledger is append-only; balance matches the sum of ledger entries.
- [ ] Concurrency test passes (only one of 50 parallel redeems succeeds).
- [ ] Runs with docker-compose.

### M2. Security

- [ ] JWT login, refresh, and USER and ADMIN roles.
- [ ] Idempotency keys on earn and redeem, with the repeat-request test passing.
- [ ] Validation, safe errors, rate limiting and audit log in place.
- [ ] Security tests pass.

### M3. Campaigns and patterns

- [ ] Multiplier, fixed bonus and category bonus rules via Strategy and Factory.
- [ ] Points expiry job with oldest-first redemption.
- [ ] OpenAPI docs and GitHub Actions CI green.

### M4. Kubernetes with 3 replicas

- [ ] Service runs as 3 replicas on Docker Desktop Kubernetes with probes, ConfigMap, Secret and limits.
- [ ] Redis cache, shared idempotency, ShedLock and SSE via Pub/Sub all work across replicas.
- [ ] Rolling update with zero failed requests.

### M5. Events and resilience

- [ ] Transactional outbox and RabbitMQ relay; notification consumer is idempotent.
- [ ] Resilience4j on the mock partner call.

### M6. Proof and polish

- [ ] Grafana dashboard, correlation IDs, k6 results with cache on and off.
- [ ] README with architecture diagram, setup steps and design decisions.
- [ ] Optional: deploy to AWS, then the frontend.

## Optional frontend (later)

A small Next.js app, built only after M4, whose job is to make the backend easy to demo in a screen share.

- User view: log in, see balance updating live over SSE, simulate a purchase, redeem points, view history.
- Admin view: create and edit campaigns, make a manual adjustment, browse the audit log.
- Demo button: fire several redeem requests at once to show the concurrency protection working.

The frontend never contains business logic; every rule is enforced by the backend.

## Design decisions and interview talking points

The README gets one or two lines per decision: the problem, the choice, and the honest trade-off.

| Decision | Why | Trade-off to admit |
|---|---|---|
| Append-only ledger | Full history, easy audits and reversals, like real payment systems | More rows; balance needs caching or a stored total |
| Optimistic locking for redeem | No lock held while waiting; fast when conflicts are rare | Retries under heavy contention; compare with SELECT … FOR UPDATE |
| Idempotency keys | Network retries must never award or spend points twice | Extra storage and a lookup on every write |
| Redis cache | Reads far outnumber writes | Risk of stale data; solved by evicting on write |
| Transactional outbox | No lost events if the app crashes mid-request | Small delay before events go out; at-least-once delivery means idempotent consumers |
| RabbitMQ or Kafka | Demonstrates event-driven design | Overkill at this scale; added to show the pattern |

| Decision | Why | Trade-off to admit |
|---|---|---|
| SSE over WebSockets | Updates only go server to client; simpler and works over plain HTTP | One-way only; needs Pub/Sub across replicas |
| Kubernetes locally | Shows replicas, probes, config and rolling updates | Not a production cluster; databases would be managed services in real life |

Questions to be ready for, answered from your own code:

1. What happens when two redeem requests arrive at the same time?
2. What happens if the client times out and retries a redeem?
3. How do you keep the cache correct across 3 replicas?
4. What happens if the app crashes after the database commit but before the event is sent?
5. How do you stop one user from reading another user's points?
6. How would you scale this to Rakuten's traffic, and what would break first?